

Esta apresentação foi criada e adaptada pelos professores Diogo Cezar Teixeira Batista, Willian Watanabe e Francisco Pereira Junior (Thesko).

Laboratório de Informática

Prof. Francisco Pereira Junior (Thesko)

Universidade Tecnológica Federal do Paraná | Engenharia de Software / TADS
Laboratório de Informática | 2020/01 | versão-1.0



MINISTÉRIO DA
EDUCAÇÃO



Linguagem C

Funções

Prof. Francisco Pereira Junior (Thesko)



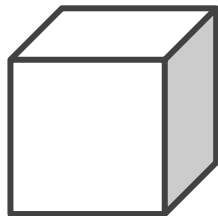
MINISTÉRIO DA
EDUCAÇÃO



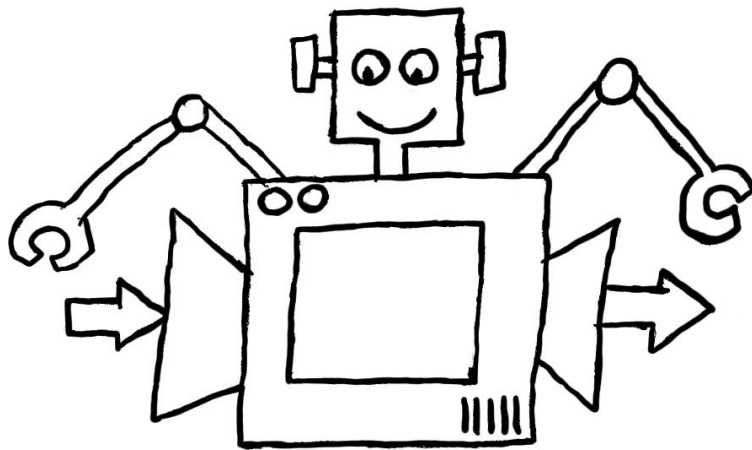
FUNÇÕES

- funções são subrotinas (blocos de código) que executam *tarefas independentes*.
- função é um conjunto de comandos agrupados em um bloco, que recebe um nome e através deste pode ser invocado.

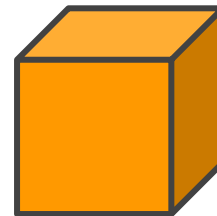
FUNÇÕES



peça sem pintura



esse é um robô pintor, sua **função** é receber uma peça sem pintura, pintar e devolver esta peça pintada



peça pintada

FUNÇÕES

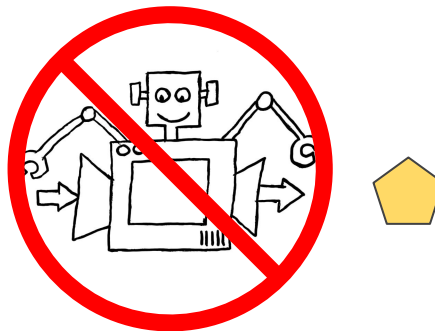
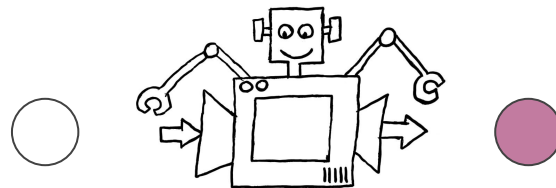
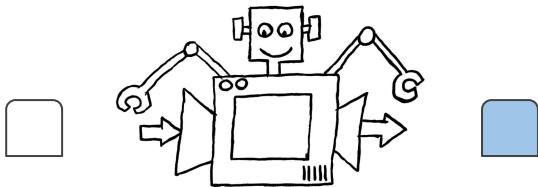
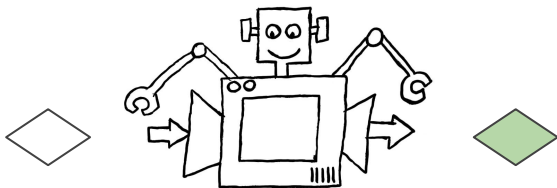
- uma função pode ser utilizada em vários programas diferentes (reutilização).
- de maneira geral:
 - seu objetivo deve estar *bem* definido (fazer o mínimo de coisa possível);
 - baseado em uma determinada entrada, suas saídas devem fornecer sempre o *resultado esperado*.

FUNÇÕES

permite que cada **partes** do seu programa seja **escrito, testado** e **revisado** individualmente, sem alterar o funcionamento do programa como um todo

FUNÇÕES

se o objetivo da função for pintar uma peça...



FUNÇÕES

- a estrutura é muito semelhante ao que já estudamos;
 - *bloco de comandos* → o que a função faz (instruções executadas pela função);
 - *parâmetros* → o que a função recebe (conforme a necessidade);
 - *retorno* → o que a função devolve (conforme a necessidade);

ESTRUTURA DE UMA FUNÇÃO

```
<tipo de retorno> nome_da_função(<parâmetros>){  
    bloco de comandos;  
    <retorno>;  
}
```

EXEMPLO PRÁTICO

o código a seguir é uma função definida pelo usuário para calcular a soma de dois números reais, e uma função principal que faz sua invocação

FUNÇÃO MAIN (principal)

```
int main() {  
    float num1, num2;  
    printf("Digite dois números:");  
    scanf("%f %f", &num1, &num2);  
    ...  
    soma(num1, num2);  
    ...  
    return 0;  
}
```

FUNÇÃO SOMA

```
void soma(float a, float b) {  
    float total;  
    total = a + b;  
    printf("%.2f", total);  
}
```

TIPOS DE FUNÇÕES

- uma função criada pelo usuário *pode ou não retornar um valor e pode ou não receber argumentos (parâmetros)*, quando for chamada;
- são 4 combinações:
 - função **sem** retorno e **sem** parâmetro;
 - função **com** retorno e **sem** parâmetro;
 - função **sem** retorno e **com** parâmetro;
 - função **com** retorno e **com** parâmetro.

FUNÇÃO SEM RETORNO E SEM PARÂMETRO

função SEM retorno e SEM parâmetro

FUNÇÃO myAvg

```
void myAvg(void) {  
    float num1 = 3.45,  
          num2 = 96.55,  
          avg = 0;  
    avg = (num1 + num2) / 2.0;  
    printf("Media = %.2f", avg);  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    ...  
    myAvg( );  
}
```

FUNÇÃO SEM RETORNO E SEM PARÂMETRO

função **SEM retorno** e SEM parâmetro

FUNÇÃO myAvg

```
void myAvg(void) {  
    float num1 = 3.45,  
          num2 = 96.55,  
          avg = 0;  
    avg = (num1 + num2) / 2.0;  
    printf("Media = %.2f", avg);  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    ...  
    myAvg( );  
}
```

FUNÇÃO SEM RETORNO E SEM PARÂMETRO

função SEM retorno e SEM parâmetro

FUNÇÃO myAvg

```
void myAvg(void) {  
    float num1 = 3.45,  
          num2 = 96.55,  
          avg = 0;  
    avg = (num1 + num2) / 2.0;  
    printf("Media = %.2f", avg);  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    ...  
    myAvg( );  
}
```

FUNÇÃO COM RETORNO E SEM PARÂMETRO

função **COM retorno** e SEM parâmetro

FUNÇÃO myAvg

```
float myAvg(void) {  
    float num1 = 3.45,  
          num2 = 96.55,  
          avg = 0;  
    avg = (num1 + num2) / 2.0;  
    return avg;  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    float var = 0;  
    ...  
    var = myAvg( );  
    printf("Média = %.2f", var);  
}
```

FUNÇÃO COM RETORNO E SEM PARÂMETRO

função COM retorno e SEM parâmetro

FUNÇÃO myAvg

```
float myAvg(void) {  
    float num1 = 3.45,  
          num2 = 96.55,  
          avg = 0;  
    avg = (num1 + num2) / 2.0;  
    return avg;  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    float var = 0;  
    ...  
    var = myAvg( );  
    printf("Média = %.2f", var);  
}
```

FUNÇÃO SEM RETORNO E COM PARÂMETRO

função **SEM retorno** e COM parâmetro

FUNÇÃO myAvg

```
void myAvg(float a, float b) {  
    float med = 0;  
    avg = (a + b) / 2.0;  
    printf("Média = %.2f", avg);  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    float num1 = 3.45,  
          num2 = 96.55,  
          ...  
    myAvg(num1, num2);  
}
```

FUNÇÃO SEM RETORNO E COM PARÂMETRO

função SEM retorno e COM parâmetro

FUNÇÃO myAvg

```
void myAvg(float a, float b) {  
    float med = 0;  
    avg = (a + b) / 2.0;  
    printf("Média = %.2f", avg);  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    float num1 = 3.45,  
          num2 = 96.55,  
          ...  
    myAvg(num1, num2);  
}
```

FUNÇÃO COM RETORNO E COM PARÂMETRO

função **COM retorno** e COM parâmetro

FUNÇÃO myAvg

```
float myAvg(float a, float b) {  
    float avg;  
    avg = (a + b) / 2.0;  
    return avg;  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    float num1 = 3.45,  
          num2 = 96.55,  
          var = 0;  
    ...  
    var = myAvg(num1, num2);  
    printf("Media = %f", var);  
}
```

FUNÇÃO COM RETORNO E COM PARÂMETRO

função COM retorno e COM parâmetro

FUNÇÃO myAvg

```
float myAvg(float a, float b) {  
    float avg;  
    avg = (a + b) / 2.0;  
    return(avg);  
}
```

FUNÇÃO MAIN (exemplo de chamada para a função myAvg)

```
void main() {  
    float num1 = 3.45,  
          num2 = 96.55,  
          var = 0;  
  
    ...  
    var = myAvg(num1, num2);  
    printf("Media = %f", var);  
}
```

LOCALIZAÇÃO DAS FUNÇÕES

- basicamente 2 posições → **antes** ou **depois** do programa principal (*main*);

antes

```
int soma(int n1, int n2) {  
    return (n1 + n2);  
}
```

```
int main(){  
    printf("%d", soma(5, 15));  
    return 0;  
}
```

depois

```
int main(){  
    printf("%d", soma(5, 15));  
    return 0;  
}
```

```
int soma(int n1, int n2) {  
    return (n1 + n2);  
}
```

OBS: (**depois**) o *main* não reconhece a função *soma*.

LOCALIZAÇÃO DAS FUNÇÕES

- basicamente 2 posições → **antes** ou **depois** do programa principal (*main*);

antes
<pre>int soma(int n1, int n2) { return (n1 + n2); }</pre>
<pre>int main(){ printf("%d", soma(5, 15)); return 0; }</pre>



depois (com protótipo)
<pre>int soma(int n1, int n2);</pre>
<pre>int main(){ printf("%d", soma(5, 15)); return 0; }</pre>
<pre>int soma(int n1, int n2) { return (n1 + n2); }</pre>



OBS: (**antes**) com muitas funções, nem sempre conseguirá ordená-las conforme a necessidade.
(**depois**) sempre garantido por conta do protótipo da função.

PROTOTIPAÇÃO DE FUNÇÕES

- Declaração de uma função (cabeçalho da função com um ; no final).
- Garante ao compilador que essas funções serão reconhecidas pelo programa.

CRIANDO PROTÓTIPOS

```
double fatorial(int n);
```

```
int main(){  
    double vl;  
    vl = fatorial(5);  
    ...  
}  
// aqui
```

```
double fatorial(int n) {  
    double calc = 1, i;  
    if (n == 0 || n == 1) {  
        return 1;  
    } else {  
        if (n > 1) {  
            for (i = 2; i <= n; i++) {  
                calc *= i;  
            }  
            return calc;  
        } else {  
            return -1;  
        }  
    }  
}
```

REGRAS DE ESCOPO PARA VARIÁVEIS

- como sabemos, uma variável só pode ser usada depois de declarada;
- onde as variáveis são reconhecidas?
- o **local** onde a variável é declarada → indica o seu **escopo**;

REGRAS DE ESCOPO PARA VARIÁVEIS

- uma variável pode ser:
 - **local** → declarada e válida apenas dentro de um bloco de comandos e/ou uma função.
 - **global** → declarada fora do bloco de uma função. Seu valor pode ser modificado por qualquer função do programa.
 - **formal** → variável local declarada na lista de parâmetros de uma função e por isso, tem validade apenas dentro da função.

REGRAS DE ESCOPO PARA VARIÁVEIS

CRIANDO PROTÓTIPOS

```
#include<stdio.h>
```

```
float calc(int n1, int n2);
```

```
float taxa = 0.5;
```

variável global - todos enxergam

```
int main(){
```

```
    float res = 0;
```

variável local - só vale dentro da função *main*

```
    res = calc(5, 15);
```

```
    printf("%.2f\n", res);
```

```
    return 0;
```

```
}
```

```
float calc(int n1, int n2) {
```

variável formal - só vale dentro da função *calc*

```
    float total = 0;
```

variável local - só vale dentro da função *calc*

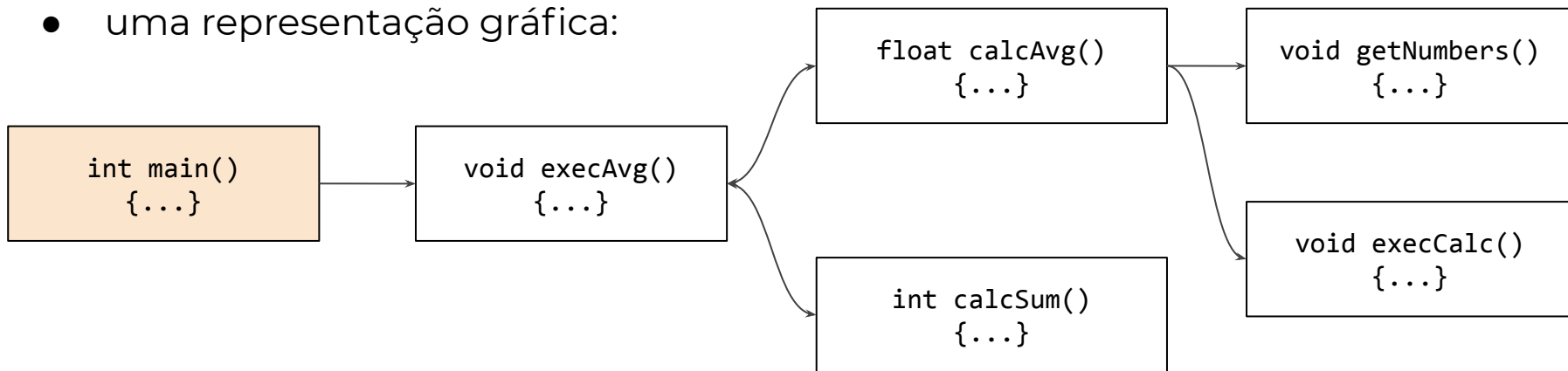
```
    total = (n1 + n2) * taxa;
```

```
    return total;
```

```
}
```

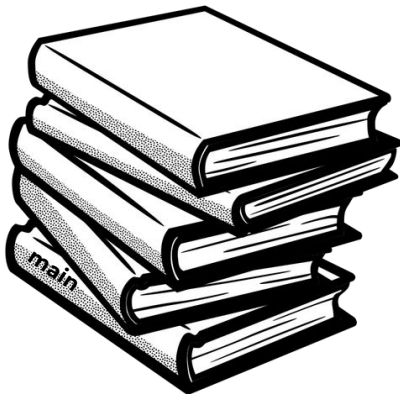
ENCADEAMENTO DE FUNÇÕES

- as funções podem ser encadeadas, ou seja, uma função pode chamar outra que chama outra e sucessivamente;
- uma representação gráfica:



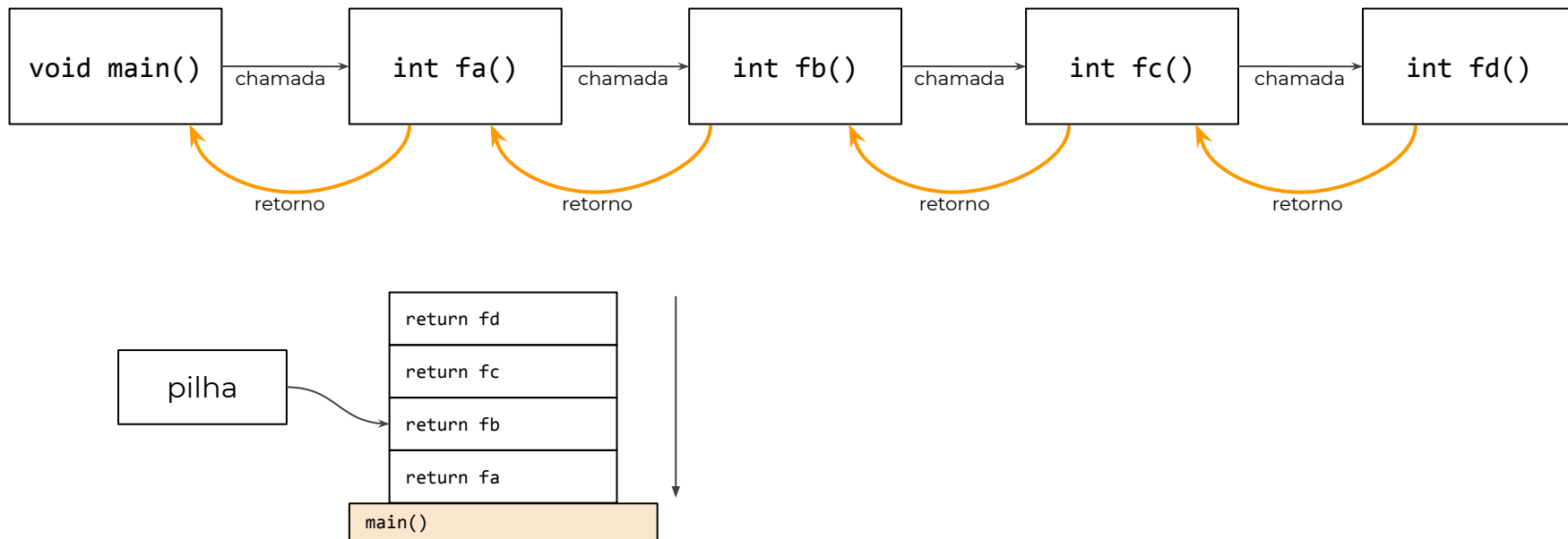
PILHA DE EXECUÇÃO

- sempre que uma função chama outra, temos uma inserção em uma **pilha de execução**;
- mas o que é uma pilha?
- podemos pensar em uma pilha de livros:



PILHA DE EXECUÇÃO

quando uma função invoca uma segunda função, o seu *retorno* fica armazenado em uma pilha:



DICAS PARA UTILIZAR FUNÇÕES

- *por onde começar?*
 - procure ler o problema primeiro e pensar como este problema poderia ser dividido em partes funcionais;
 - por exemplo, muitos problemas precisam de um programa que leia dados do usuário, faça cálculos e os apresente na tela. Então, é comum dividir o programa em uma parte de leitura, uma para o cálculo e outra para apresentação;
 - se cada parte ainda ficar muito grande ou se essa parte gerar mais de um dado como resultado, então ela deve ser dividida em mais partes;
 - uma vez que você tenha dividido o problema em partes, pense também como estas partes devem se relacionar;

DICAS PARA UTILIZAR FUNÇÕES

- *consegui pensar nas funções que meu programa terá e sei como relacioná-las. O que faço agora?*
 - uma boa ideia é definir como cada função irá realizar sua tarefa;
 - por exemplo, se a função serve para ler um dado do usuário, defina as instruções de printf e scanf.

DICAS PARA UTILIZAR FUNÇÕES

- *as funções que criei precisam de parâmetros?*
 - uma vez que você saiba como a função deve realizar sua tarefa, você deve se perguntar: há algum dado a ser usado nesta função que foi definido *fora* dela? Se sim, você precisa de um parâmetro;

DICAS PARA UTILIZAR FUNÇÕES

- *as funções que criei precisam retornar algum valor?*
 - para responder a esta pergunta você deve se perguntar: há algum dado definido nesta função que será usado fora dela (em outra função ou no programa principal)?
 - Se sim, é preciso retornar um valor.
 - Se não, provavelmente você deverá imprimir esse valor.
 - e se eu perceber que há mais de um dado definido nesta função e que serão usados em outro lugar (em outra função)? Então, você deve dividir esta função em duas (ou mais).

DICAS PARA UTILIZAR FUNÇÕES

- *criei cada função. Agora como faço para relacioná-las?*
 - tudo começa na função main (principal);
 - a partir dela, você chamará suas novas funções criadas, conforme a necessidade;
 - se a função chamada precisa de um dado externo, então você precisa passar esse valor como argumento;
 - se a função a ser chamada retornar um valor, então, sempre atribua o valor de retorno à uma variável (de tipo compatível);

PASSANDO VETORES E MATRIZES PARA FUNÇÕES

para passar os vetores ou matrizes podemos utilizar:

CRIANDO PROTÓTIPOS

```
void callMat(int x[][5]);  
void callVet(int x[]);
```

```
int main(){  
    int a[10], b[5][5];  
    ...  
    callMat(b);  
    callVet(a);  
    ...  
    return 0;  
}
```

para matriz, ao menos
uma dimensão é
obrigatória

Esta apresentação foi criada e adaptada pelos professores Diogo Cezar Teixeira Batista, Willian Watanabe e Francisco Pereira Junior (Thesko).

Laboratório de Informática

Prof. Francisco Pereira Junior (Thesko)

Universidade Tecnológica Federal do Paraná | Engenharia de Software / TADS
Laboratório de Informática | 2020/01 | versão-1.0



MINISTÉRIO DA
EDUCAÇÃO

